For example, let's analyze the text shown in the next listing, which defines *dosa*—a South Indian savory pancake. The code analyzes a description of this dish using a `fingerprint` analyzer.

Listing 7.15 Analyzing text using a `fingerprint` analyzer

```
POST _analyze
{
  "text": "A dosa is a thin pancake or crepe originating from South India.
  ➥ It is made from a fermented batter consisting of lentils and rice.",
  "analyzer": "fingerprint"
}
```

When we execute this code, the output is as follows:

```
"tokens" : [{
  "token" : "a and batter consisting crepe dosa fermented from india is it
  ➥ lentils made of or originating pancake rice south thin",
  "start_offset" : 0,
  "end_offset" : 130,
  "type" : "fingerprint",
  "position" : 0
  }]
```

When you look closely at the response, you see that the output consists of only one token. The words were lowercased and sorted, duplicate words (*a*, *of*, *from*) were removed, and the set of words was turned into a single token. Any time we have text that needs to be deduplicated, sorted, and concatenated, `fingerprint` is the logical choice.

7.3.6 The pattern analyzer

Sometimes we want to tokenize and analyze text based on a certain pattern—for example, removing the first *n* numbers of a phone number or removing the dash between every four digits of a credit card number. Elasticsearch provides a `pattern` analyzer for that purpose.

The default `pattern` analyzer splits sentences into tokens based on non-word characters. This pattern is represented as `\W+` internally. As figure 7.11 shows, the `pattern` tokenizer and the `lowercase` and `stop` filters form the `pattern` analyzer.

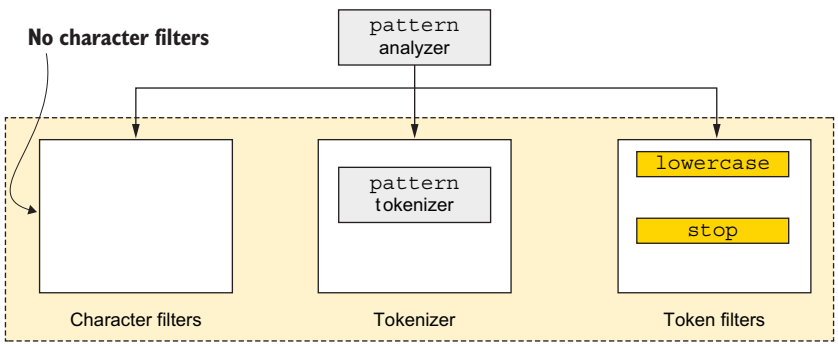


Figure 7.11 Anatomy of the `pattern` analyzer